

Comparação Experimental de Algoritmos de Classificação Aplicados ao Dataset Diamonds

José Yrikes de Oliveira Feitosa¹

¹ Engenharia de Computação

Universidade Federal Rural de Pernambuco – Unidade Acadêmica de Belo Jardim (UFRPE/UABJ)

Disciplina: Inteligência Artificial – Professor: Anderson Cavalcanti

Belo Jardim – PE – Brasil

Abstract. *This paper compares supervised classification algorithms applied to the Diamonds dataset, using diamond cut quality as the target variable. Decision Tree, KNN, Naive Bayes, Logistic Regression, Linear SVM and MLP were evaluated with preprocessing pipelines, stratified 10-fold cross-validation and grid search. The results are analyzed using macro-averaged metrics, class imbalance, stability across folds and computational feasibility. Decision Tree achieved the best overall balance between performance and generalization, while MLP showed the most balanced class-level behavior.*

Resumo. *Este artigo compara algoritmos supervisionados de classificação aplicados ao dataset Diamonds, utilizando a qualidade do corte do diamante como variável alvo. Foram avaliados Árvore de Decisão, KNN, Naive Bayes, Regressão Logística, SVM Linear e MLP com pipelines de pré-processamento, validação cruzada estratificada com 10 folds e busca em grade. Os resultados são analisados por métricas macro, desbalanceamento das classes, estabilidade entre folds e viabilidade computacional. A Árvore de Decisão obteve o melhor equilíbrio geral entre desempenho e generalização, enquanto a MLP apresentou o comportamento mais equilibrado por classe.*

1. Introdução

A classificação é uma tarefa central do aprendizado de máquina supervisionado, na qual o objetivo é associar instâncias de entrada a classes previamente conhecidas. Em problemas reais, diferentes algoritmos podem apresentar comportamentos distintos conforme a distribuição das classes, a escala dos atributos, a presença de variáveis categóricas e a complexidade das relações entre os dados.

Neste trabalho, investiga-se a classificação da qualidade do corte de diamantes a partir do dataset Diamonds. A variável alvo é *cut*, composta pelas classes Fair, Good, Very Good, Premium e Ideal. Foram comparados seis algoritmos: Árvore de Decisão, KNN, Naive Bayes, Regressão Logística, SVM Linear e MLP.

O objetivo é avaliar o desempenho desses algoritmos sob o mesmo procedimento experimental, utilizando validação cruzada estratificada com 10 folds, busca de hiperparâmetros por GridSearchCV e métricas macro, especialmente o F1-score macro, devido ao desbalanceamento da variável alvo. O artigo está organizado em fundamentação teórica, metodologia experimental, resultados, discussão e considerações finais.

O restante deste artigo está organizado da seguinte forma: a Seção 2 apresenta a fundamentação teórica dos algoritmos utilizados; a Seção 3 descreve a metodologia experimental; a Seção 4 apresenta os resultados obtidos; a Seção 5 discute os resultados; e a Seção 6 apresenta as considerações finais.

2. Fundamentação Teórica e Trabalhos Relacionados

Esta seção apresenta uma breve descrição dos algoritmos de classificação utilizados no experimento. Todos os modelos foram implementados com a biblioteca scikit-learn, que fornece uma interface padronizada para pré-processamento, treinamento, validação cruzada e avaliação de modelos de aprendizado de máquina [Pedregosa et al. 2011]. A comparação envolveu classificadores com diferentes características: modelos baseados em regras, distância, probabilidade, fronteiras lineares e redes neurais.

2.1. Árvore de Decisão

Árvores de decisão são modelos supervisionados que realizam divisões sucessivas no espaço dos atributos, formando uma estrutura hierárquica de decisão. Cada nó interno representa uma regra de separação, enquanto as folhas representam as classes previstas. Esse tipo de modelo é interpretável e consegue capturar relações não lineares, mas pode sofrer sobreajuste quando a árvore cresce excessivamente [Breiman et al. 1984, scikit-learn developers 2026b].

No experimento, foram testadas diferentes configurações para critério de divisão, profundidade máxima, número mínimo de amostras por folha, número mínimo de amostras para divisão e pesos de classe. Esses parâmetros são importantes porque controlam o grau de complexidade da árvore e sua capacidade de generalização.

2.2. K-Vizinhos Mais Próximos

O algoritmo K-Vizinhos Mais Próximos, ou KNN, classifica uma instância com base nas classes dos exemplos mais próximos no espaço de atributos. A decisão depende da quantidade de vizinhos considerada, da métrica de distância e do peso atribuído aos vizinhos. Por ser baseado em distância, o KNN é sensível à escala dos atributos, tornando necessária a padronização das variáveis numéricas [Cover and Hart 1967, scikit-learn developers 2026c].

No experimento final, foi utilizada uma grade reduzida para o KNN, variando o número de vizinhos e mantendo a distância euclidiana com ponderação por distância. Essa escolha reduziu o custo computacional sem deixar de atender ao requisito mínimo de combinações de hiperparâmetros.

2.3. Naive Bayes

Naive Bayes é uma família de classificadores probabilísticos baseados no Teorema de Bayes. O método assume independência condicional entre os atributos, hipótese que simplifica o cálculo das probabilidades. Apesar de ser uma suposição forte e nem sempre verdadeira em dados reais, o Naive Bayes é frequentemente usado como modelo de referência por ser simples, rápido e computacionalmente eficiente [scikit-learn developers 2026f].

Neste trabalho, foi utilizado o `GaussianNB`, adequado para atributos numéricos contínuos. O hiperparâmetro avaliado foi `var_smoothing`, que controla a suavização

da variância utilizada no cálculo das probabilidades. Como o dataset Diamonds possui atributos potencialmente correlacionados, como dimensões, quilate e características de qualidade, esperava-se que a hipótese de independência pudesse limitar o desempenho desse modelo.

2.4. Regressão Logística

A Regressão Logística é um modelo linear utilizado para classificação. Embora o nome contenha o termo regressão, o modelo estima probabilidades de pertencimento às classes e define fronteiras de decisão lineares. Em problemas multiclasse, pode ser aplicada por estratégias de generalização adequadas. A regularização, controlada pelo parâmetro C , é importante para reduzir sobreajuste e controlar a complexidade do modelo [James et al. 2023, scikit-learn developers 2026d].

No experimento, a Regressão Logística foi avaliada com diferentes valores de C e com ou sem uso de pesos de classe. Essa configuração permitiu observar tanto o efeito da regularização quanto o impacto do desbalanceamento no desempenho do classificador.

2.5. Máquina de Vetores de Suporte

Máquinas de Vetores de Suporte, ou SVM, buscam construir uma fronteira de decisão que maximize a margem entre as classes. O método pode utilizar diferentes funções de kernel para representar fronteiras lineares ou não lineares. Entretanto, kernels não lineares podem ter custo computacional elevado em datasets maiores [Cortes and Vapnik 1995, scikit-learn developers 2026i].

Neste trabalho, foi utilizada como modelo principal uma versão linear por meio do `LinearSVC`. Essa escolha reduziu o custo computacional em relação a configurações com kernels mais pesados, como o RBF. Foram avaliados diferentes valores de C e o uso ou não de `class_weight`. Adicionalmente, foi realizado um experimento complementar com kernel RBF para verificar se a limitação do SVM Linear estava associada à restrição de fronteiras lineares.

2.6. Redes Neurais MLP

O Perceptron Multicamadas, ou MLP, é uma rede neural artificial composta por camadas de neurônios. O treinamento ocorre por retropropagação do erro, com ajuste iterativo dos pesos. O MLP pode capturar relações não lineares, mas é sensível à escala dos dados, à arquitetura da rede, à taxa de aprendizado e à regularização [Rumelhart et al. 1986, Goodfellow et al. 2016, scikit-learn developers 2026e].

No experimento, foi utilizada uma configuração leve de MLP para manter o custo computacional viável. Foram testadas três arquiteturas de camadas ocultas, mantendo os demais parâmetros principais fixos. Como o modelo depende de otimização numérica, o escalonamento dos atributos foi mantido no pipeline.

3. Metodologia Experimental

A metodologia foi estruturada para garantir reprodutibilidade, evitar vazamento de dados durante o pré-processamento e atender à validação cruzada estratificada com 10 folds.

3.1. Dataset Utilizado

O conjunto de dados utilizado foi o Diamonds, disponibilizado no Kaggle. O dataset contém informações sobre diamantes, incluindo atributos físicos, atributos de qualidade e preço. A tarefa definida neste trabalho foi a classificação da variável `cut`, que representa a qualidade do corte do diamante. As informações gerais do dataset são apresentadas na Tabela 1.

Tabela 1. Descrição geral do dataset Diamonds

Característica	Valor
Nome do dataset	Diamonds
Fonte	Kaggle
Identificador	ulrikthgepedersen/diamonds
Tarefa	Classificação multiclasse
Variável alvo	cut
Número de exemplos	53.940
Número de colunas originais	10
Número de atributos usados em X	8
Número de classes	5
Classe majoritária	Ideal
Exemplos da classe majoritária	21.551
Percentual da classe majoritária	39,95%

A variável alvo apresenta desbalanceamento: a classe `Ideal` representa 39,95% dos exemplos, enquanto `Fair` representa apenas 2,98%. Por isso, a avaliação priorizou métricas macro, especialmente o F1-score macro, que atribui o mesmo peso médio a cada classe.

Tabela 2. Distribuição das classes da variável alvo

Classe	Quantidade	Percentual
Ideal	21.551	39,95%
Premium	13.791	25,57%
Very Good	12.082	22,40%
Good	4.906	9,10%
Fair	1.610	2,98%

A Tabela 2 evidencia a predominância das classes `Ideal`, `Premium` e `Very Good`, além da baixa frequência de `Fair`. Essa diferença de frequência justifica o uso de métricas macro e a análise do efeito das estratégias de balanceamento.

3.2. Pré-processamento dos Dados

O pré-processamento foi realizado com preservação de uma cópia do dataset original. Não foram encontrados valores ausentes. Foram detectadas 146 linhas duplicadas, mantidas no experimento pela configuração `REMOVE_DUPLICADAS=False`. Também foi verificada a presença de dimensões inválidas nas colunas `x`, `y` e `z`, mas nenhuma linha foi removida por esse critério.

A coluna `price` foi removida dos atributos preditores para evitar que o modelo dependesse diretamente de uma variável comercial fortemente associada à qualidade percebida do diamante. Após a separação entre atributos e alvo, o conjunto de entrada ficou com 53.940 exemplos e 8 atributos preditores: seis numéricos e dois categóricos.

Tabela 3. Resumo do pré-processamento aplicado

Etapa	Procedimento adotado
Limpeza inicial	Verificação de nulos, duplicatas e dimensões inválidas.
Variável removida	<code>price</code> , para reduzir dependência de informação comercial.
Atributos numéricos	<code>carat</code> , <code>depth</code> , <code>table</code> , <code>x</code> , <code>y</code> , <code>z</code> .
Atributos categóricos	<code>color</code> , <code>clarity</code> , codificados por <code>OneHotEncoder</code> .
Pipeline	Uso de <code>ColumnTransformer</code> , imputadores e <code>StandardScaler</code> quando necessário.

As transformações foram implementadas dentro de pipelines, garantindo que o pré-processamento fosse ajustado apenas nos dados de treino de cada fold, evitando vazamento de informação [scikit-learn developers 2026g, scikit-learn developers 2026h].

3.3. Validação Cruzada e Busca de Hiperparâmetros

O procedimento experimental utilizou uma divisão inicial estratificada, reservando 20% dos dados para teste final. O conjunto de busca ficou com 43.152 exemplos e o conjunto de teste com 10.788 exemplos. A seleção dos modelos foi realizada por `GridSearchCV` com validação cruzada estratificada de 10 folds, `shuffle=True` e `random_state=42`. A métrica principal de seleção foi o F1-score macro.

Tabela 4. Algoritmos e hiperparâmetros efetivamente avaliados

Modelo	Hiperparâmetros avaliados	Comb.
Árvore de Decisão	<code>criterion</code> : gini, entropy; <code>max_depth</code> : None, 5, 10, 20; <code>min_samples_leaf</code> : 1, 5, 10; <code>min_samples_split</code> : 2, 10; <code>class_weight</code> : None, balanced.	96
KNN	<code>n_neighbors</code> : 5, 11, 21, 45, 151; <code>weights</code> : distance; <code>p</code> : 2.	5
Naive Bayes	<code>var_smoothing</code> : 10^{-9} , 10^{-8} , 10^{-7} .	3
Regressão Logística	<code>C</code> : 0.1, 1, 10; <code>class_weight</code> : None, balanced.	6
SVM Linear	<code>C</code> : 0.1, 1, 10; <code>class_weight</code> : None, balanced.	6
MLP	<code>hidden_layer_sizes</code> : (32), (64), (32,16); <code>alpha</code> : 0.0001; <code>learning_rate_init</code> : 0.001.	3

As grades tiveram tamanhos diferentes por restrições computacionais, mas todos os modelos atenderam ao requisito mínimo de três combinações de hiperparâmetros. Modelos sensíveis à escala, como KNN, Regressão Logística, SVM e MLP, utilizaram `StandardScaler`. Para lidar com desbalanceamento, foram avaliados pesos de classe nos modelos compatíveis e oversampling quando aplicável. Acurácia, acurácia balanceada, precisão macro, revocação macro e F1-score macro foram coletadas para análise [scikit-learn developers 2026a].

A Figura 1 complementa a Tabela 2: além de mostrar o comportamento das métricas, sintetiza a lógica do balanceamento no treino e as diferenças entre os modelos quanto ao uso de escalonamento, oversampling e pesos de classe.

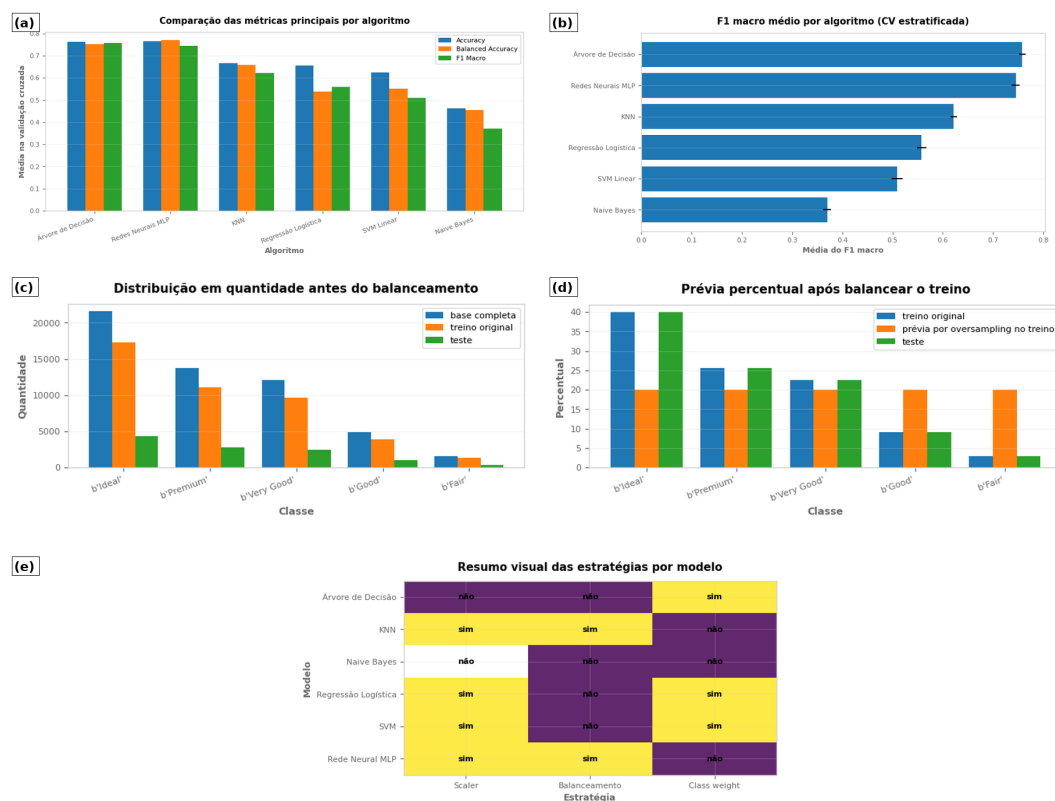


Figura 1. Painel metodológico e analítico: métricas principais por algoritmo, F1 macro médio, distribuição antes do balanceamento, prévia após balancear o treino e estratégias adotadas por modelo.

4. Resultados

A Tabela 5 apresenta o melhor resultado de cada algoritmo segundo o F1-score macro. Os valores de validação cruzada são apresentados como média e desvio padrão entre os 10 folds, e a última coluna mostra o F1 macro no conjunto de teste final.

Tabela 5. Desempenho geral dos melhores modelos

Modelo	Acc. CV	Bal. Acc. CV	Prec. Macro CV	F1 Macro CV	F1 Teste
Árvore de Decisão	0,7636 ± 0,0046	0,7522 ± 0,0092	0,7660 ± 0,0055	0,7580 ± 0,0068	0,7584
MLP	0,7643 ± 0,0078	0,7691 ± 0,0088	0,7292 ± 0,0070	0,7453 ± 0,0078	0,7422
KNN	0,6650 ± 0,0063	0,6581 ± 0,0065	0,6098 ± 0,0078	0,6213 ± 0,0068	0,6243
Regressão Logística	0,6549 ± 0,0049	0,5372 ± 0,0076	0,6487 ± 0,0119	0,5576 ± 0,0088	0,5472
SVM Linear	0,6237 ± 0,0062	0,5511 ± 0,0101	0,5094 ± 0,0083	0,5089 ± 0,0110	0,4952
Naive Bayes	0,4614 ± 0,0068	0,4534 ± 0,0103	0,4008 ± 0,0066	0,3697 ± 0,0079	0,3822

A Árvore de Decisão obteve o maior F1 macro no teste final, com 0,7584, e manteve resultado praticamente igual ao da validação cruzada. Essa proximidade indica boa estabilidade entre a etapa de seleção de hiperparâmetros e a avaliação final. A MLP apresentou desempenho muito próximo, com F1 macro de 0,7422 no teste final, além de acurácia balanceada ligeiramente superior, o que sugere melhor recuperação média das classes.

O KNN ficou em posição intermediária, com F1 macro de 0,6243. Embora tenha

vido estável, seu desempenho ficou abaixo dos modelos não lineares mais flexíveis, possivelmente pela sensibilidade ao espaço de atributos expandido após a codificação one-hot. Regressão Logística e SVM Linear tiveram resultados inferiores, sinalizando limitação de fronteiras lineares para separar as categorias de `cut`. O Naive Bayes teve o menor desempenho, funcionando como referência probabilística simples.

As melhores configurações incluíram Árvore de Decisão com `criterion=gini` e regularização por `min_samples_leaf=10`; MLP com 64 neurônios na camada oculta; KNN com 151 vizinhos e ponderação por distância; Regressão Logística com $C = 10$; SVM Linear com $C = 1$ e `class_weight=balanced`; e Naive Bayes com `var_smoothing=10-7`. As combinações completas foram exportadas pelo notebook.

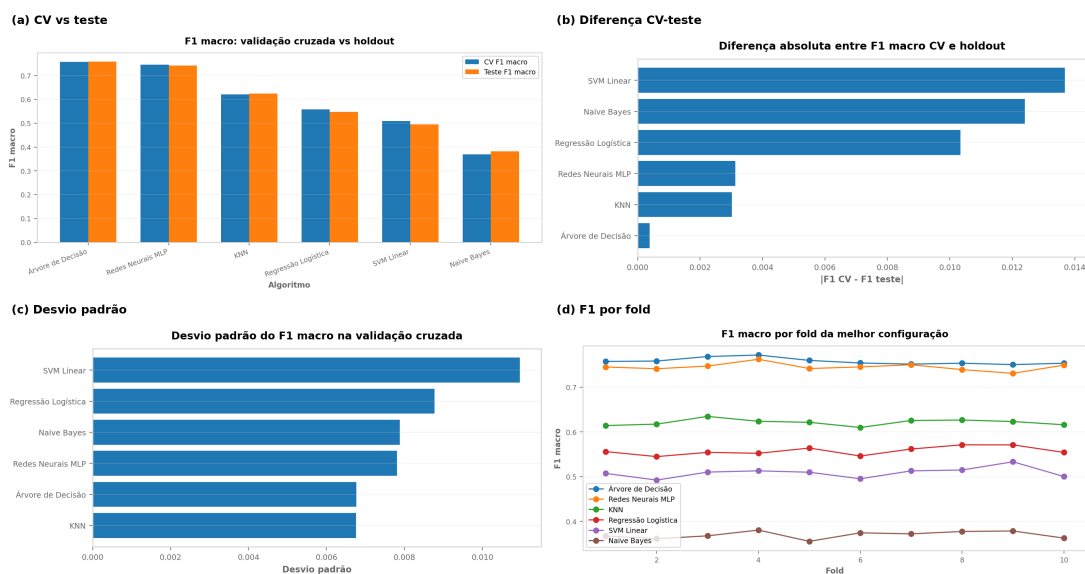


Figura 2. Painel de desempenho e estabilidade: F1 macro na validação cruzada e no teste final, diferença entre validação e teste, desvio padrão entre folds e variação do F1 macro por fold.

A Figura 2 mostra que Árvore de Decisão e MLP permaneceram no topo, mas por razões diferentes: a Árvore apresentou menor diferença entre validação cruzada e teste final, enquanto a MLP manteve desempenho competitivo com maior equilíbrio médio entre classes. A baixa diferença entre validação e teste nos melhores modelos reduz indícios de sobreajuste severo.

A Figura 3 evidencia que a dificuldade do problema não está apenas na baixa frequência de algumas classes. A classe `Very Good`, apesar de não ser a menor classe, concentrou parte importante dos erros e confusões, sugerindo sobreposição com categorias vizinhas de qualidade. Esse resultado é coerente com a natureza ordinal da variável `cut`, cujas classes representam gradações de qualidade e não grupos completamente independentes.

5. Discussão Crítica dos Resultados

A análise dos resultados não deve considerar apenas o maior F1-score macro. Como o dataset apresenta desbalanceamento de classes e categorias de qualidade próximas entre

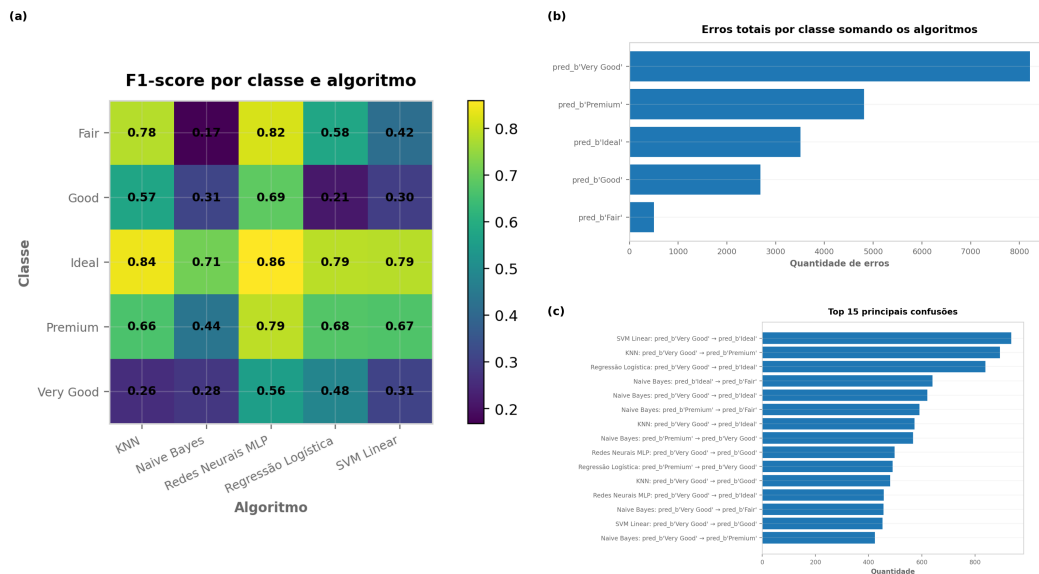


Figura 3. Painel de análise por classe: F1-score por classe e algoritmo, erros totais por classe e principais confusões entre classes.

si, também é necessário observar a estabilidade entre validação cruzada e teste final, o comportamento por classe, o impacto do pré-processamento e o custo computacional das escolhas feitas.

5.1. Desempenho Geral e Estabilidade

A Árvore de Decisão apresentou o melhor equilíbrio geral entre desempenho, estabilidade e generalização. O modelo obteve F1 macro de 0,7580 na validação cruzada e 0,7584 no teste final, uma diferença praticamente nula. Esse resultado é importante porque indica que o desempenho obtido durante a seleção de hiperparâmetros se manteve no conjunto separado para teste, reduzindo indícios de sobreajuste severo.

A MLP ficou muito próxima da Árvore de Decisão, com F1 macro de 0,7453 na validação cruzada e 0,7422 no teste final. Embora tenha ficado abaixo da Árvore no F1 macro final, a MLP apresentou acurácia balanceada superior, o que sugere melhor recuperação média entre as classes. Assim, a escolha entre Árvore e MLP não é apenas uma questão de maior valor numérico: a Árvore foi mais estável e interpretável, enquanto a MLP apresentou comportamento mais equilibrado por classe.

O KNN ocupou uma posição intermediária, com F1 macro de 0,6243 no teste final. Esse resultado mostra que o algoritmo conseguiu capturar parte da estrutura dos dados, mas ficou distante dos modelos não lineares mais flexíveis. Como o KNN depende diretamente da noção de vizinhança no espaço de atributos, seu desempenho pode ter sido afetado pela codificação one-hot das variáveis categóricas, que aumenta a dimensionalidade e altera as relações de distância entre as instâncias.

Regressão Logística e SVM Linear tiveram desempenho inferior aos modelos não lineares. Como ambos constroem fronteiras lineares no espaço de atributos transformado, seus resultados sugerem que a variável `cut` depende de relações mais complexas entre atributos físicos e qualitativos. Essa interpretação é reforçada pelo experimento comple-

mentar com SVM RBF, que obteve F1 macro de $0,7166 \pm 0,0044$ na validação cruzada e 0,7261 no teste final. Portanto, o baixo desempenho do SVM Linear não indica que o SVM seja inadequado para o problema, mas que a configuração linear foi limitada para representar a separação entre as classes.

O Naive Bayes apresentou o menor desempenho, com F1 macro de 0,3822 no teste final. Esse comportamento era esperado em parte, pois o método assume independência condicional entre os atributos. No dataset Diamonds, atributos como quilate, dimensões, cor, clareza e corte tendem a estar relacionados, o que enfraquece a hipótese central do modelo. Ainda assim, o Naive Bayes cumpre um papel útil como referência probabilística simples e de baixo custo computacional.

5.2. Análise por Classe e Sobreposição entre Categorias

A análise por classe mostra que o problema não se resume ao desbalanceamento. A classe `Fair` é a menos frequente, mas a classe `Very Good` também apresentou grande dificuldade de separação, apesar de possuir quantidade muito maior de exemplos. Isso indica que parte dos erros ocorre porque algumas categorias de corte estão próximas no espaço de atributos, e não apenas porque há poucas amostras em determinada classe.

Esse ponto é relevante porque a variável `cut` possui uma natureza ordinal: `Fair`, `Good`, `Very Good`, `Premium` e `Ideal` representam gradações de qualidade. Entretanto, neste trabalho, o problema foi tratado como classificação multiclasse nominal, isto é, sem informar explicitamente ao modelo que há uma ordem entre as classes. Assim, erros entre classes vizinhas, como `Good` e `Very Good`, ou entre `Very Good` e `Premium`, podem ser mais compreensíveis do que erros entre classes muito distantes.

A Figura 3 reforça essa interpretação ao mostrar o F1-score por classe, os erros totais e as principais confusões. A MLP apresentou o comportamento mais equilibrado entre as classes, enquanto a Árvore de Decisão obteve o melhor desempenho global. Essa diferença mostra que o melhor modelo depende do critério de escolha: se o objetivo for desempenho geral e estabilidade, a Árvore é mais adequada; se o objetivo for equilíbrio entre classes, a MLP se torna uma alternativa competitiva.

5.3. Impacto do Pré-processamento

O pré-processamento teve papel central no experimento. A remoção da variável `price` reduziu o risco de o modelo depender excessivamente de uma informação comercial fortemente associada à qualidade percebida do diamante. Essa decisão torna a tarefa mais difícil, pois remove uma variável potencialmente preditiva, mas deixa a avaliação mais alinhada ao objetivo de investigar padrões físicos e qualitativos dos diamantes.

A codificação one-hot permitiu representar os atributos categóricos `color` e `clarity` em formato adequado para os modelos do scikit-learn. Entretanto, essa transformação também aumenta a dimensionalidade dos dados. Esse aumento pode prejudicar especialmente o KNN, que depende de distâncias, e os modelos lineares, que passam a separar as classes em um espaço expandido. Já a Árvore de Decisão tende a lidar melhor com esse tipo de estrutura, pois não depende diretamente de escala nem de distância entre pontos.

O escalonamento foi necessário para KNN, Regressão Logística, SVM e MLP. No KNN, atributos em escalas diferentes poderiam dominar o cálculo de distância. Na

Regressão Logística, no SVM e na MLP, a padronização contribui para a estabilidade da otimização numérica. A Árvore de Decisão, por outro lado, não depende diretamente de escalonamento, o que ajuda a explicar sua robustez em dados tabulares como este.

5.4. Efeito do Balanceamento

O balanceamento teve efeito desigual entre os algoritmos. A classe `Ideal` representa 39,95% do dataset, enquanto `Fair` representa apenas 2,98%, justificando o uso de métricas macro e estratégias como `class_weight="balanced"` ou `oversampling`. Entretanto, os resultados mostram que balancear os dados ou usar pesos de classe não garante automaticamente melhor desempenho global.

Na Árvore de Decisão e na Regressão Logística, as melhores configurações não utilizaram `class_weight="balanced"`, indicando que a ponderação das classes não foi a melhor escolha para maximizar o F1 macro nesses modelos. No SVM Linear, por outro lado, a melhor configuração utilizou `class_weight="balanced"`, sugerindo que os pesos ajudaram a compensar parcialmente o desbalanceamento.

Nos modelos em que a pipeline permitiu `oversampling`, como KNN, Naive Bayes e MLP, o balanceamento pode ter ajudado a expor melhor as classes menos frequentes durante o treinamento. Porém, a persistência de erros em classes intermediárias mostra que o principal obstáculo não foi apenas a frequência das classes, mas também a sobreposição entre categorias próximas. Portanto, o balanceamento foi uma estratégia importante, mas não suficiente para resolver completamente o problema.

Como não foi realizada uma ablação controlada comparando cada algoritmo com e sem balanceamento mantendo todos os demais hiperparâmetros fixos, não é possível atribuir os ganhos de desempenho exclusivamente ao balanceamento. Essa é uma limitação importante da interpretação experimental.

5.5. Custo Computacional e Limitações

O custo computacional variou bastante entre os algoritmos. A Árvore de Decisão realizou 960 ajustes devido à grade mais ampla de hiperparâmetros. O KNN realizou menos combinações, mas possui custo de predição maior, pois depende do cálculo de distâncias. Regressão Logística e SVM Linear tiveram custo moderado. O Naive Bayes foi o modelo mais leve. A MLP foi mantida com uma grade reduzida para preservar viabilidade computacional.

A escolha do SVM Linear também foi motivada por custo computacional. O resultado superior do SVM RBF mostra que kernels não lineares poderiam melhorar o desempenho, mas também aumentariam o tempo de execução. Isso evidencia uma troca comum em aprendizado de máquina: modelos mais expressivos podem capturar melhor a estrutura dos dados, mas exigem maior custo de treinamento e ajuste.

As principais limitações do experimento foram: a busca de hiperparâmetros limitada por tempo e recursos, o tratamento de `cut` como variável nominal apesar de sua natureza ordinal, a remoção de `price`, que reduz dependência comercial mas também retira informação preditiva, e o aumento de dimensionalidade causado pela codificação one-hot. Mesmo com essas limitações, os resultados permitem comparar de forma consistente os algoritmos exigidos, pois todos foram avaliados com o mesmo procedimento de validação e com métricas adequadas ao desbalanceamento.

6. Considerações Finais

Este trabalho comparou seis algoritmos de classificação aplicados ao dataset Diamonds, utilizando pipelines de pré-processamento, validação cruzada estratificada com 10 folds e busca de hiperparâmetros por `GridSearchCV`. A métrica principal foi o F1-score macro, devido ao desbalanceamento das classes.

Os resultados indicaram que modelos capazes de capturar relações não lineares apresentaram melhor desempenho. A Árvore de Decisão obteve o melhor equilíbrio geral entre desempenho, estabilidade e generalização, enquanto a MLP apresentou resultado próximo e melhor comportamento por classe. O KNN teve desempenho intermediário, mas foi afetado pela dimensionalidade e pela sobreposição entre categorias próximas. Regressão Logística e SVM Linear ficaram abaixo dos modelos não lineares, indicando limitação das fronteiras lineares para este problema. O Naive Bayes apresentou o menor desempenho, possivelmente pela violação da hipótese de independência condicional entre atributos.

A análise complementar com SVM RBF reforçou que o baixo desempenho do SVM Linear não deve ser interpretado como limitação geral do método SVM, mas como consequência da configuração linear adotada. Esse resultado mostra que a escolha de kernel e hiperparâmetros pode alterar significativamente o desempenho em problemas com relações não lineares.

Como trabalhos futuros, sugere-se ampliar a busca de hiperparâmetros, avaliar modelos ensemble, testar estratégias adicionais de balanceamento, investigar o impacto da variável `price` na modelagem e considerar abordagens de classificação ordinal para explorar a ordem natural das classes de qualidade do corte.

Referências

- Breiman, L., Friedman, J. H., Olshen, R. A., and Stone, C. J. (1984). *Classification and Regression Trees*. Wadsworth International Group.
- Cortes, C. and Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20:273–297.
- Cover, T. M. and Hart, P. E. (1967). Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1):21–27.
- Goodfellow, I., Bengio, Y., and Courville, A. (2016). *Deep Learning*. MIT Press.
- James, G., Witten, D., Hastie, T., Tibshirani, R., and Taylor, J. (2023). *An Introduction to Statistical Learning: With Applications in Python*. Springer.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, E. (2011). Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12:2825–2830.
- Rumelhart, D. E., Hinton, G. E., and Williams, R. J. (1986). Learning representations by back-propagating errors. *Nature*, 323:533–536.
- scikit-learn developers (2026a). Classification metrics. Acesso em: 29 jun. 2026.
- scikit-learn developers (2026b). Decisiontreeclassifier. Acesso em: 29 jun. 2026.

scikit-learn developers (2026c). Kneighborsclassifier. Acesso em: 29 jun. 2026.

scikit-learn developers (2026d). Logisticregression. Acesso em: 29 jun. 2026.

scikit-learn developers (2026e). Mlpclassifier. Acesso em: 29 jun. 2026.

scikit-learn developers (2026f). Naive bayes. Acesso em: 29 jun. 2026.

scikit-learn developers (2026g). Pipeline. Acesso em: 29 jun. 2026.

scikit-learn developers (2026h). Preprocessing data. Acesso em: 29 jun. 2026.

scikit-learn developers (2026i). Support vector machines. Acesso em: 29 jun. 2026.